

INFORME FINAL UNIFICADO DE ANÁLISIS DE VULNERABILIDADES

SHAREfactory (CUSA00572) – Módulo `ffmpeg_prx.prx` y `libc.prx`

Fecha: 30 de julio de 2026

Versión: 1.0 (consolidada)

Investigador: H4SS9M

1. Resumen Ejecutivo

Se ha realizado un análisis estático exhaustivo de los binarios `ffmpeg_prx.prx` y `libc.prx` de la aplicación SHAREfactory (versión 03.02, CUSA00572) para PlayStation 4 con firmware 11.00. Se identificaron **cinco vulnerabilidades de corrupción de memoria** (cuatro heap overflows y un stack overflow), de las cuales **solo una es directamente accesible desde un archivo MP4 malicioso: el heap overflow en la concatenación de la configuración del codec H.264 (avcC)**. Las demás residen en demuxers de formatos no soportados por SHAREfactory (ASF, AVI, FLV) o están protegidas por stack canary.

El heap overflow accesible es reproducible y permite controlar el tamaño del desbordamiento, pero su explotación final para ejecución de código está bloqueada por la falta de información sobre la disposición exacta de los objetos en el heap. Se ha confirmado la presencia de **safe unlinking** en el asignador de memoria (`libc.prx`), lo que impide ataques de unlink clásicos. Se han extraído **gadgets ROP** del binario que permitirían construir una cadena de ejecución una vez conseguido el control del flujo.

La conclusión unánime es que se requiere un **volcado de memoria del proceso** (obtenido mediante un payload externo cargado con GoldHEN) para determinar los offsets precisos y completar el exploit.

2. Binarios Analizados

Archivo	MD5	Descripción
<code>ffmpeg_prx.prx</code>	<code>aaa1ddf3784ce3205be4b41ed04fb493</code>	Motor FFmpeg (enero 2018) con demuxers, codecs y parsers. Cargado en <code>0x80284000</code> .

Archivo	MD5	Descripción
libc.prx	(interno)	Biblioteca C del sistema con dlmalloc (mspace). Cargado en 0x80268000.

3. Herramientas Utilizadas

- **IDA Pro 9.1** con decompilador Hex-Rays y scripts IDAPython personalizados.
- **Python 3** para generación de archivos MP4 mutados, fuzzing y captura de klogs.
- **PS4 FW 11.00** con **GoldHEN** (carga de payloads, BinLoader).
- **Capturador inteligente de klog** (script Python) para registrar y comparar crashes.

4. Funciones Clave y Referencias Cruzadas

4.1 Funciones principales del parser

Función	Dirección (IDA)	Dirección runtime	Rol
sub_610FB0	0x610FB0	0x802841B0 *	malloc interno del mspace.
sub_5E74E0	0x5E74E0	0x8026B4E0 *	Copia de datos desde el flujo (avio_read).
sub_595A60	0x595A60	0x80279A60 *	Manejador de hijos de udta (contiene stack overflow en covr).
sub_171583	0x171583	0x80275583 *	Concatena extradata H.264 – causa el heap overflow explorable.
sub_171A70	0x171A70	0x80275A70 *	Parsea SPS/PPS del avcC .
sub_1715D0	0x1715D0	0x802755D0 *	Llama a sub_171A70 y sub_171583 .
sub_305200	0x305200	0x80289200 *	avcodec_open2 – inicializa el códec.
sub_1E70	0x1E70	0x80265E70 *	Hilo LoadVideoTracksThread . Aquí ocurre el crash (mov rax, [rcx+8] con rcx=0).

Nota: Las direcciones runtime marcadas con asterisco se calculan sumando la base 0x80284000 a los offsets de IDA. Verificar los offsets exactos en IDA puede requerir ajustes si

el mapeo no es lineal; se recomienda usar las direcciones del klog como referencia definitiva.

4.2 Funciones de otros demuxers (no accesibles desde MP4)

Función	Dirección IDA	Formato	¿Accesible?
sub_559C40	0x559C40	AVI (avi_read_packet)	No
sub_551B30	0x551B30	ASF	No
sub_571EC0	0x571EC0	FLV (flv_read_packet)	No

4.3 Función de verificación de safe unlinking

Función	Dirección IDA	Ubicación	Descripción
sub_660A0	0x660A0	libc.prx	Comprobación fd->bk == chunk && bk->fd == chunk antes de desvincular un chunk libre.

5. Vulnerabilidades Encontradas

5.1 Heap Overflow en Concatenación de avcC (Accesible)

Ubicación: sub_171583 en ffmpeg_prx.prx (0x171583).

Tipo: Heap Buffer Overflow (CWE-122).

Disparador: Archivo MP4 con un átomo avcC que contenga más datos de los que el buffer original puede alojar.

Causa raíz:

Al inicializar el codec H.264, la función sub_171583 concatena el nuevo extradata (SPS/PPS) sobre el buffer existente sin reasignar memoria. El puntero al buffer avanza (v9 += v14) pero el espacio reservado sigue siendo el original. La copia posterior sub_168 escribe fuera del límite, corrompiendo el heap.

Código vulnerable (decompilado de sub_171583):

```
v9 = *(unsigned __int8 **)(v4 + v8 + 64);    // buffer original
if ( a3 ) {
    v10 = *(_QWORD *)(v3 - 344);
    v11 = *(_QWORD *)(v4 + v8 + 64);
    *(_QWORD *)(v3 - 256) = 0;
```

```

result = sub_171A70(v10, v11, (unsigned __int64 *)(v3 - 256), a1);
LODWORD(v13) = *(_DWORD *)(v6 + 32); // tamaño original
v14 = *(_QWORD *)(v3 - 256); // nuevo tamaño
v9 += v14; // ¡avanza sin reasignar!
*(_QWORD *)(v4 + v8 + 72) = v14 + (int)v13;
}
sub_168(v9, *(_QWORD *)(v6 + 24), (int)v13); // overflow

```

Xrefs relevantes:

- sub_171A70 ← llamada desde sub_171583 (0x1715D0).
- sub_171583 ← llamada desde sub_1715D0.
- sub_1715D0 ← invocado durante la inicialización del codec H.264 dentro de sub_305200.
- sub_305200 ← llamado desde sub_1E70 (0x1E70+278/397).

Efecto observado:

El crash ocurre en sub_1E70 (mov rax, [rcx+8] con rcx=0) porque el array de streams del AVFormatContext (_R12+48) se corrompe a NULL. Las pruebas de heap feng shui (spray de pistas, hole filling) no consiguieron colocar un objeto con vtable adyacente.

Limitación actual:

Se desconoce la distancia exacta desde el buffer desbordado hasta un puntero de función (vtable de codec, array de streams, etc.). Sin esa información no es posible redirigir la ejecución.

5.2 Stack Overflow en el Manejador de covr (Protegido)

Ubicación: sub_595A60 (0x595A60).

Tipo: Stack Buffer Overflow (CWE-121).

Disparador: Átomo covr con tipo de imagen 13 (JPEG) o 27 (PNG) y tamaño > 96 bytes.

Código vulnerable:

```

if ( a3 == 1920364387 ) { // 'covr'
    v56 = v52 - 16;
    a2 = (const char *)&v234; // buffer local de 96 bytes ([rsp+460h])
    a1 = (__int64)v223;
    v14 = sub_5E74E0((__int64)v223, (__int64)&v234, v56);
}

```

Protección:

La función posee stack canary (if (*(_QWORD *)sub_0 == v238) al final). El overflow avanza hacia direcciones altas, por lo que no alcanza variables críticas situadas por debajo de v234

(como el puntero a función `v227`). El canario se corrompe y el proceso aborta sin control de `rip`.

Limitación: Stack canary + disposición desfavorable de las variables locales.

5.3 Heap Overflow en Demuxer AVI (No Alcanzable)

Función: `sub_559C40` (`avi_read_packet`, `0x559C40`).

Tipo: Heap Overflow.

Disparador: Chunk `movi` con tamaño de paquete excesivo.

Código vulnerable:

```
v39 = 0x7FFFFFFF;  
v40 = *(_DWORD*)(v81 + 24);  
if ( v40 >= 2 ) {  
    v39 = v40 << 10;  
    if ( v40 >= 32 )  
        v39 = *(_DWORD*)(v81 + 24);  
}  
if ( v39 > *(_DWORD*)(v81 + 8) )  
    v39 = *(_DWORD*)(v81 + 8);  
v9 = sub_5E74E0(v92, v83, v39); // v83 es AVPacket*, no pkt->data
```

Nota: La copia se hace directamente sobre la estructura `AVPacket`, permitiendo sobrescribir campos internos.

Limitación: SHAREfactory no abre archivos AVI.

5.4 Heap Overflow en Demuxer ASF (No Alcanzable)

Función: `sub_551B30` (`0x551B30`).

Disparador: Campo "Secret Data" con tamaño enorme.

Limitación: Formato ASF no soportado.

5.5 Heap Overflow en Demuxer FLV (No Alcanzable)

Función: `sub_571EC0` (`flv_read_packet`, `0x571EC0`).

Disparador: Paquetes de video/audio con tamaños inconsistentes.

Limitación: Formato FLV no soportado.

6. Mecanismos de Protección

Mecanismo	Ubicación	Efecto
Safe Unlinking	sub_660A0 en libc.prx	Bloquea ataques de unlink clásicos (comprobación <code>fd->bk == chunk && bk->fd == chunk</code>).
Stack Canary	sub_595A60 y otras	Impide sobrescritura directa de la dirección de retorno en el stack.

7. Gadgets ROP Identificados

Todos los gadgets se encuentran en `ffmpeg_prx.prx`. Las direcciones se muestran como offset en IDA y como dirección runtime (base `0x80284000` + offset).

Gadget	Offset IDA	Dirección runtime	Bytes
<code>pop rdi; ret</code>	<code>0x01bc11</code>	<code>0x8029FC11</code>	<code>5f c3</code>
<code>pop rsi; ret</code>	<code>0x039841</code>	<code>0x802BD841</code>	<code>5e c3</code>
<code>pop rdx; ret</code>	<code>0x0b828c</code>	<code>0x8033C28C</code>	<code>5a c3</code>
<code>pop rax; ret</code>	<code>0x016eda</code>	<code>0x8029AEDA</code>	<code>58 c3</code>
<code>pop rcx; ret</code>	<code>0x0162d4</code>	<code>0x8029A2D4</code>	<code>59 c3</code>
<code>pop rbx; ret</code>	<code>0x0110ae</code>	<code>0x802950AE</code>	<code>5b c3</code>
<code>pop rbp; ret</code>	<code>0x0040b6</code>	<code>0x802880B6</code>	<code>5d c3</code>
<code>pop rsp; ret</code>	<code>0x00542f</code>	<code>0x8028942F</code>	<code>5c c3</code>
<code>leave; ret</code>	<code>0x1f3006</code>	<code>0x80417006</code>	<code>c9 c3</code>
<code>ret</code>	<code>0x00403c</code>	<code>0x8028803C</code>	<code>c3</code>

Nota: Las direcciones runtime pueden variar ligeramente según la versión exacta del binario; se recomienda verificar con el klog la base real de `ffmpeg_prx.prx`.

8. Pruebas Dinámicas Realizadas

- **Archivo** `mutado_race.mp4` **original:** crash en `0x802865AF` (`rcx=0`). Confirmado heap overflow en la pista.
- **Eliminación del hijo corrupto** (`meta` **vacío**): el crash persiste → la causa está en la pista, no en `meta`.
- `minimal.mp4` (**pista de video correcta, sin** `avcC` **corrupto**): no crashea → aísla el fallo en el `avcC`.
- **Inyección del** `avcC` **original en** `minimal.mp4`: crashea de nuevo → confirma el bug en la concatenación de extradata.
- **Manipulación del tamaño de SPS/PPS en** `avcC`: variaciones en `rbp` y `r12`, pero sin control de `rcx` o `rip`.
- **Pruebas de heap spray (múltiples pistas, hole filling)**: no lograron adyacencia con una `vtable`.
- **Intentos de activar demuxers AVI/ASF/FLV desde MP4**: formatos no reconocidos por SHAREfactory.

9. Diagrama de Flujo del Crash (Heap Overflow en `avcC`)

1. SHAREfactory abre un MP4 con un `avcC` malicioso.
2. `sub_171583` concatena el nuevo extradata → **overflow**.
3. El heap se corrompe, dañando estructuras internas.
4. Al llegar a `sub_1E70`, el array de streams (`_R12+48`) es NULL.
5. `rcx = 0` → `mov rax, [rcx+8]` → **SIGSEGV**.

10. Limitaciones y Próximos Pasos

Problema central: El heap overflow en `avcC` es el único explotable desde un MP4, pero sin conocer la disposición del heap no se puede calcular el offset necesario para sobrescribir un puntero de función. Las protecciones (safe unlinking, stack canary) descartan ataques más simples.

Solución propuesta: Obtener un **volcado de memoria del proceso** `eboot.bin` justo después del overflow. Con ese volcado se podrá:

- Localizar el buffer de extradata y los objetos adyacentes.
- Medir la distancia a un puntero de función (`vtable` de codec, `callback` de `AVCodecContext`, etc.).
- Construir un payload preciso con los gadgets ROP ya identificados.

Método para el volcado:

- Usar un payload precompilado de volcado de memoria para PS4 11.00 (ej. `ps4debug`, `PS4-11.00-Dumper`), cargado mediante el **BinLoader** de GoldHEN.
 - Si no se encuentra uno funcional, compilar el payload propio con OpenOrbis (se requiere configurar correctamente la toolchain en Windows).
-

11. Conclusión

Se ha completado un análisis exhaustivo que ha identificado **cinco vulnerabilidades reales** en los binarios de SHAREfactory, con solo una de ellas accesible desde el vector de ataque (archivos MP4). La explotación final está bloqueada por la falta de información sobre la disposición del heap en tiempo de ejecución. La obtención de un volcado de memoria es el paso crítico restante para lograr la ejecución de código arbitrario. Se dispone de los gadgets ROP necesarios y de un conocimiento profundo de las funciones implicadas, por lo que una vez superado este obstáculo, el exploit podrá ser completado con alta probabilidad de éxito.